

CargoRoute: Robot Path Planning and Optimization for Warehouse Logistics

Zerui Wu

Department of Electrical and Computer Engineering
UC Santa Barbara
Santa Barbara, USA
zeruiwu@ucsb.edu

Chun Yu Chang

Department of Electrical and Computer Engineering
UC Santa Barbara
Santa Barbara, USA
cchang973@ucsb.edu

Abstract—This project studies route planning for a single robot that collects all cargo in a warehouse scenario. In order to avoid obstacles, we combine visibility graph and Dijkstra algorithm to define the distance between two cargo instead of using Euclidean distance. We focus on two problems: an unweighted single tour, modeled as a traveling salesman problem (TSP), and a weighted version with a weight capacity that splits the work into capacity-feasible tours, modeled as a capacitated vehicle routing problem (CVRP). For TSP problem, we use NetworkX and Gurobi to solve it. For CVRP problem, we use OR-Tools and Gurobi to solve it. The results show that we can get the optimal solution of a single-tour TSP problem efficiently while it's computationally heavy for CVRP problem to find the optimal solution.

I. INTRODUCTION

A warehouse robot that collects cargo must decide the order in which to visit its pickup points. If its load is limited, it must also decide how to divide the work into separate tours. These are combinatorial routing problems that become costly to solve exactly as the number of pickups grows.

We study one robot serving 39 fixed cargo points from a depot. Ignoring weight gives a single-tour traveling salesman problem (TSP); adding a weight capacity Q splits the route into capacity-feasible tours, which is formulated as a capacitated vehicle routing problem (CVRP). Both run on a visibility-graph distance matrix that routes around the shelves instead of through them. For each problem we pair a fast heuristic with an exact solver, NetworkX with 2-opt against Gurobi for the TSP and OR-Tools against Gurobi for the CVRP, to see where exact optimization is worth its runtime.

II. MODEL FORMULATION

A. Scenario Formulation

A single robot collects cargo in a 23×28 warehouse and returns to a depot at the origin. Shelves form five rows: A and B hold five upright shelves each, C, D, and E hold three sideways shelves each. A fixed set of 39 cargo points lies in the aisles, with no randomization, so results are reproducible. The scenario is shown in Fig. 1.

Both models need pairwise distances between two pickup points, and a straight line may cross a shelf. We inflate each shelf into a rectangle by a small safety margin and reformulate it as an obstacle region, remap every cargo point to its nearest free aisle cell, and build a visibility graph over the depot,

pickup points, and obstacle corners, with an edge wherever the segment clears all shelf interiors. All-pairs Dijkstra then gives the obstacle-aware distance d_{ij}^{VG} and its waypoint path. The graph matrix using d_{ij}^{VG} as the distances between each two nodes is the only geometric input; each model outputs a route to gather all the cargo and return to the depot.

B. Traveling Salesman Problem

The traveling salesman problem (TSP) looks for the shortest closed tour that visits every node exactly once and returns to the start. Because the distances d_{ij}^{VG} are symmetric ($d_{ij}^{VG} = d_{ji}^{VG}$), we use the undirected formulation. We use $V = \{0, 1, 2, \dots, n\}$ ($n = 39$ in our scenario) to denote the set of routing nodes, where node 0 is the depot. For each unordered pair of nodes we define a binary variable

$$x_{ij} = \begin{cases} 1, & \text{if edge } \{i, j\} \text{ is on the tour,} \\ 0, & \text{otherwise,} \end{cases} \quad \forall i, j \in V, i < j, \quad (1)$$

and minimize the total tour distance, using the obstacle-aware costs d_{ij}^{VG} from Section II-A:

$$\begin{aligned} \min_x \quad & \sum_{\substack{i, j \in V \\ i < j}} d_{ij}^{VG} x_{ij} \\ \text{s.t.} \quad & \sum_{\substack{j \in V \\ j \neq i}} x_{ij} = 2, \quad \forall i \in V, \\ & \sum_{\substack{i, j \in S \\ i < j}} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| \leq |V| - 1, \\ & x_{ij} \in \{0, 1\}, \quad \forall i, j \in V, i < j \end{aligned} \quad (2)$$

Because the TSP is formulated on an undirected graph, edge $\{i, j\}$ and edge $\{j, i\}$ represent the same connection. For notational simplicity, we write $x_{ij} = x_{ji}$ when expressing the degree constraints. The constraint $\sum_{j \in V, j \neq i} x_{ij} = 2$ ensures that the solution must form a closed cycle that visits every node exactly once, the robot should exactly enter and leave each node once. However, the degree constraints alone are not sufficient to guarantee a single tour without disconnected

cycles. Therefore, the second constraint is used to eliminate subtours. For any proper subset $S \subset V$, the number of selected edges whose two endpoints both lie in S must be at most $|S| - 1$. This prevents the nodes in S from forming an independent closed cycle disconnected from the rest of the tour.

C. Traveling Salesman Problem with Weight Constraint

The weighted scenario extends the TSP with cargo weights and a weight capacity, so the route splits into several capacity-feasible tours (CVRP). The mathematical formulation of traveling salesman problem is shown as (3):

$$\begin{aligned}
\min_{x,z,y} \quad & \sum_{k \in K} \sum_{\substack{i,j \in V \\ i \neq j}} d_{ij}^{VG} x_{ij}^k \\
\text{s.t.} \quad & \sum_{k \in K} z_i^k = 1, \quad \forall i \in C, \\
& \sum_{\substack{j \in V \\ j \neq i}} x_{ij}^k = z_i^k, \quad \forall k \in K, \forall i \in C, \\
& \sum_{\substack{j \in V \\ j \neq i}} x_{ji}^k = z_i^k, \quad \forall k \in K, \forall i \in C, \\
& \sum_{j \in C} x_{0j}^k = y_k, \quad \forall k \in K, \\
& \sum_{i \in C} x_{i0}^k = y_k, \quad \forall k \in K, \\
& \sum_{i \in C} w_i z_i^k \leq Q \cdot y_k, \quad \forall k \in K, \\
& \sum_{k \in K} y_k = L, \\
& \sum_{i \in S} \sum_{\substack{j \in S \\ j \neq i}} x_{ij}^k \leq |S| - 1, \quad \forall k \in K, \forall S \subseteq C, |S| \geq 2, \\
& x_{ij}^k \in \{0, 1\}, \quad \forall k \in K, \forall i, j \in V, i \neq j, \\
& z_i^k \in \{0, 1\}, \quad \forall k \in K, \forall i \in C, \\
& y_k \in \{0, 1\}, \quad \forall k \in K.
\end{aligned} \tag{3}$$

where Q is the weight capacity for the robot, $C = \{1, 2, \dots, n\}$ is the set of cargo pickup nodes and $K = \{1, 2, \dots, m\}$ is the set of admissible depot-to-depot tours. In our scenario, the number of tours is fixed with the minimum number of tours L which is shown as (4).

$$L = \left\lceil \frac{\sum_{i \in C} w_i}{Q} \right\rceil \tag{4}$$

We use three binary variables to encode the routing and assignment choices, where x_{ij}^k denotes whether the robot travels directly from node i to node j in tour k , y_k indicates if tour k is active in the solution, z_i^k is value one when the robot picks cargo i in tour k .

In the CVRP problem, the objective function still minimizes the total obstacle-aware travel distance over all selected edges and all tours, while the constraints need to be adjusted due to the capacity constraint. $\sum_{k \in K} z_i^k = 1$ ensures that each cargo is assigned to an exact tour. $\sum_{\substack{j \in V \\ j \neq i}} x_{ji}^k = z_i^k$ and $\sum_{\substack{j \in V \\ j \neq i}} x_{ij}^k = z_i^k$ mean that the robot must enter and leave cargo i pickup point respectively once in tour k if cargo i is picked in tour k . $\sum_{j \in C} x_{0j}^k = y_k$ and $\sum_{i \in C} x_{i0}^k = y_k$ enforce that in each tour the robot should start from the depot and eventually return to the depot. $\sum_{i \in C} w_i z_i^k \leq Q \cdot y_k$ provides the weight capacity constraint, the total weight of each tour cannot be greater than Q . Additionally, for CVRP problem we also need the similar subtour-elimination constraint.

III. METHODOLOGY

A. Network X and 2-opt

NetworkX [1] is a Python library for building and analyzing graphs. It provides an approximate TSP solver function `traveling_salesman_problem` that returns a valid tour quickly, though the tour is a heuristic with no optimality guarantee. 2-opt is a local search optimization that eliminates overlapping segments: it removes two edges and reconnects the tour the other way, keeping the swap whenever the total length drops. Repeating this until no swap helps leaves the tour at a local optimum. We use NetworkX for the initial tour and 2-opt to refine it.

B. Gurobi with lazy constraint

Gurobi [2] is a commercial solver for mixed-integer programs. It can certify global optimality if the branch-and-bound search closes the optimality gap; otherwise, it returns the best feasible incumbent found within the time limit together with a best bound. A lazy constraint is the constraint that the solver leaves out of the initial model and adds only when a candidate solution violates it. This fits the TSP and CVRP well, since their subtour elimination constraints are too many to list up front: the solver adds each one only when a solution actually forms a subtour. Lazy constraints are implemented when using Gurobi solver to improve its computational speed.

C. OR-Tools from Google

OR-Tools [3] is an open-source optimization library developed by Google. Its routing module solves vehicle routing problems by using metaheuristics to produce a good solution quickly. Similarly to NetworkX, it is heuristic and does not prove optimality. We use OR-Tools as the fast and scalable counterpart to the Gurobi when dealing with CVRP problem.

IV. EXPERIMENT

We run four experiments, pairing a heuristic with an exact solver in each scenario. For the TSP we compare NetworkX with 2-opt against Gurobi; for the CVRP we compare OR-Tools against Gurobi.

The results of unweighted TSP are shown in Fig 1. Table I shows that for the unweighted TSP the heuristic and exact solvers are almost indistinguishable: NetworkX with 2-opt

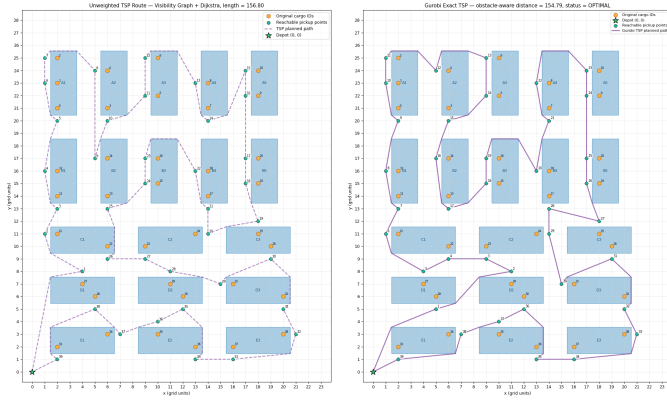


Fig. 1. The Warehouse Scenario and Comparison of TSP solution paths. The left panel shows the heuristic path generated by NetworkX (dashed line), while the right panel displays the verified optimal path solved by Gurobi (solid line).

TABLE I
TSP SOLVER COMPARISON

	Distance	Time (s)	Optimality
NetworkX	156.8	< 1s	No
Gurobi	154.79	< 1s	Yes

returns 156.8 and Gurobi returns the optimal 154.79, roughly 1.3% apart, both in one second. Although both consume less than a second, Gurobi is proven to be the optimal solution; it's the ideal choice here. The weighted CVRP behaves very differently. Table II shows the experiment results for CVRP. OR-Tools settles at 430.15 within twenty seconds and doesn't improve in 300 seconds, while Gurobi needs more than twelve hours to reach 425.07, a tour only about 1.2% shorter, the routes are shown in Fig 2; for the first nine hours it is actually worse than OR-Tools gets at one second. Both finish far from the best bound of 292.50 computed by Gurobi in 12.59 hours, with an optimality gap of over 31%, so neither result is provably optimal. In this scenario, the CVRP is computationally overwhelmed that the Gurobi solver cannot close the gap in a practical runtime, which makes the OR-Tools method a more sensible choice at this scale.

V. CONCLUSION

In both TSP and CVRP problem settings, we compared fast heuristic ways with an exact mixed-integer solver Gurobi,

TABLE II
CVRP SOLVER COMPARISON

	Incumbent Distance	Best bound	Time	MIP GAP
OR-Tools	441.56	292.50	1 s	33.8%
OR-Tools	431.26	292.50	5 s	32.2%
OR-Tools	430.15	292.50	20 s	32.0%
OR-Tools	430.15	292.50	300 s	32.0%
Gurobi	444.63	292.50	8.73 hr	34.2%
Gurobi	438.91	292.50	10.99 hr	33.4%
Gurobi	437.87	292.50	11.31 hr	33.2%
Gurobi	425.07	292.50	12.59 hr	31.2%
Best Bound	-	292.50	12.59 hr	-



Fig. 2. Gurobi CVRP Result

and the two cases pointed in opposite directions. The single-tour TSP problem can be solved efficiently, where Gurobi can compute the optimal value in one second and `traveling_salesman_problem` function in NetworkX package offers an approximate solution that is very close to the optimal solution obtained by Gurobi. In the TSP case, it's computationally feasible to get an optimal solution, so using Gurobi is an ideal choice. The CVRP case shows the opposite result: Gurobi cannot prove optimality even after twelve hours' running, while OR-Tools reaches a close result in seconds, then we can conclude that a heuristic method is more practical at this scale.

In future work, it would be valuable to explore whether there is an efficient way to find the global minimum for a problem on a larger scale, such as CVRP problem, or if there exists any method to get a more precise lower bound.

REFERENCES

- [1] A. Hagberg, D. Schult, and P. Swart, "NetworkX: traveling_salesman_problem," NetworkX documentation. [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.approximation.traveling_salesman.traveling_salesman_problem.html
- [2] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual." [Online]. Available: <https://www.gurobi.com/>
- [3] Google, "Google OR-Tools." [Online]. Available: <https://github.com/google/or-tools>